RobiS User Manual

This manual covers 2D mapping and navigation with Gmapping, Hector, Karto and Cartographer, plus the Autoware APP recording, mapping, localization, cruising and waypoint-following workflow. Commands use the deployed paths; the Autoware section uses numbered button instructions.

1 Connect to Jetson with NoMachine

Connect the computer and Jetson to the same local network. Check the current Jetson IP address on the robot’s touchscreen. In NoMachine, create an NX connection, enter the IP address shown on the touchscreen in the Host field, and set the port to 4000.

Open the connection and sign in with username jetson and password robify1234. Then select the Jetson Ubuntu desktop.

2 Robot Functions

Function	Section
Gmapping / Hector / Karto	See Three 2D Mapping and Navigation Methods
Cartographer	See Cartographer Mapping and Navigation
Autoware APP	See the Autoware APP sections
Astra Pro Plus Camera	Camera and Image Viewing
UM982 RTK	RTK Startup and Data Viewing

Run only one mapping or navigation stack at a time. Before switching, press Ctrl+C in each launch terminal and confirm the previous processes have exited. The Autoware APP starts its own sensors; do not also launch robot_start.launch.

Three 2D Mapping and Navigation Methods

Gmapping, Hector and Karto share the hardware launch and map-saving procedure. Only mapping_mode changes.

1 Start the hardware

Terminal A: enter the container, run the command, and leave it running.

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start robot_start.launch
```

2 Start one mapping algorithm

Terminal B: enter the container and select exactly one command below. RViz opens automatically by default.

Gmapping

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start 2d_mapping.launch mapping_mode:=gmapping
```

Hector

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start 2d_mapping.launch mapping_mode:=hector
```

Karto

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start 2d_mapping.launch mapping_mode:=karto
```

Drive slowly and smoothly through the area using your controller. Check that walls form consistently. Avoid abrupt turns and wheel slip, and stop extending the map if localization becomes inconsistent.

3 Save the map

Terminal C: keep mapping running, enter the container, and execute:

```
bash /home/jetson/robis/enter-navigation-gui.sh
sh "$(rospack find robot_start)/sh/2d_save_map.sh"
```

The default map files are robot_start/map/map.pgm and map.yaml. Saving again updates the default map; back up any map you want to keep first.

```
ls -lh "$(rospack find robot_start)/map/"
```

4 Switch to navigation

Press Ctrl+C in Terminal B to stop mapping and keep the hardware in Terminal A running. At the container prompt in Terminal B, run the navigation command below. Maps from Gmapping, Hector and Karto share this entry; mapping_mode is not needed.

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start 2d_navigation.launch
```

When RViz opens, use 2D Pose Estimate to set the actual position and heading. Confirm scan alignment, then use 2D Nav Goal to click a target and drag its heading. Press Ctrl+C in this terminal to stop navigation.

Cartographer Mapping and Navigation

1 Start mapping

Terminal A: enter the container and start the hardware. Do not start a second copy if it is already running.

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start robot_start.launch
```

Terminal B: enter the container and start Cartographer mapping. Its configured RViz opens automatically.

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start carto_2d_mapping.launch
```

Drive smoothly through the area and check scan-to-map alignment. Stop the robot before saving the completed map.

2 Save the occupancy map and localization state

Terminal C: while the mapper is still running, enter the container and execute:

```
bash /home/jetson/robis/enter-navigation-gui.sh
sh "$(rospack find robot_start)/sh/save_map.sh"
```

This deployment requires a matching map.pbstream, map.pgm and map.yaml for Cartographer navigation. PGM/YAML alone are insufficient for its pure-localization launch. Confirm that all three files exist and belong to the same mapping session.

```
ls -lh "$(rospack find robot_start)/map/"
```

3 Start Cartographer navigation

After saving, press Ctrl+C in Terminal B to stop mapping. Keep the hardware running, then start:

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch robot_start teb_navigation_slam.launch
```

This loads the default occupancy map and pbstream, Cartographer localization, TEB navigation and the matching RViz. Do not also run 2d_navigation.launch. The default map basename is map; when archiving or changing maps, keep the loaded files paired.

4 Check localization and send a goal

When RViz opens, use 2D Pose Estimate to click the robot’s current position on the map and drag in its actual heading. Confirm that live scans align with the mapped walls, then use 2D Nav Goal to select a reachable position and drag the desired heading.

Autoware APP Preparation and Functions

This section follows the APP buttons for the adapted robot. Stop standard 2D mapping, Cartographer navigation and other stacks first. Do not launch another copy of the sensors.

1 Open and connect the APP

Open two Jetson terminals and use the commands below to start the backend and the APP. The top-left status shows Online once connected.

Container Terminal A: start the backend.

```
bash /home/jetson/robis/enter-navigation-gui.sh
roslaunch ros_robot_control ros_robot_control.launch
```

Container Terminal B: start the APP.

```
bash /home/jetson/robis/enter-navigation-gui.sh
rosrun robot_rviz_16 robot_rviz_16
```

Use the verified ros_robot_control backend and robot_rviz_16 interface. Set Master IP to http://127.0.0.1:11311/ and Local IP to 127.0.0.1, then click Connect. Checking Auto Connect while running does not reconnect immediately; it applies on the next start.

2 Button functions

Start Pointclouds Recording

Records lidar, IMU and odometry for offline map building.

Start Mapping

Replays the recorded bag and builds an NDT point-cloud map.

Start Waypoint Saver

Records replayed or localized poses for trajectory cruising.

Start Navigation

Loads the map and starts NDT localization and TEB. It does not start cruising automatically.

Cruise Trajectory

Sends navigation goals along the saved route. Supports return trips and looping, with TEB obstacle avoidance.

Start Waypoint Follower

Uses Pure Pursuit to follow a CSV route. It is mutually exclusive with Navigation; obstacle-stop behavior must be checked separately.

The APP axes are configured for chassis frame base_link, while the global fixed frame remains map. White geometry is a PCD/map display; colored geometry is a live scan display, depending on the selected coloring settings.

Autoware Recording Mapping and Navigation

1 Stop other functions

Stop other mapping, navigation and waypoint-following functions. Keep only the backend and APP; confirm Online.

2 Start point-cloud recording

Drive smoothly forward along the intended cruising route. Remember the starting position and heading. Avoid reversing and large S-turns the robot cannot reproduce autonomously. Stop recording when finished.

3 Start Waypoint Saver before Start Mapping

Mapping replays the bag just recorded. Waypoint Saver must run before playback begins to capture replayed poses. Do not wait until mapping finishes to start it.

4 Wait for replay completion then stop Waypoint Saver

Wait until the playback Duration progress completes, then stop Waypoint Saver. Confirm that the old mapping and playback processes have exited before switching to navigation. Investigate an empty route before cruising.

5 Return to the start and start Navigation

Drive back to the recording start with approximately the same heading. Click Start Navigation. It loads localization and planning; it does not automatically drive the route.

6 Confirm localization

Open the APP RViz tab and check live-cloud alignment with the map. Use 2D Pose Estimate if position or heading is wrong. Continue only after relocalization is stable.

7 Select a single goal or trajectory cruising

Single goal: use 2D Nav Goal to choose a nearby reachable position and drag its heading. Cruising: return to Operation Panel and click Cruise Trajectory. Check Trajectory Loop for repeated return trips. Start the first cruise at the recorded start and leave room to turn at both ends.

Do not overlap mapping and navigation, or Navigation and Waypoint Follower. After stopping a function, wait for its nodes to exit to avoid duplicate TF-node shutdowns.

Astra Pro Plus Camera and Image Viewing

Run every command in this section in a Jetson Ubuntu host terminal, outside the navigation container. The workspace is /home/jetson/astra_ws and uses host ROS2 Humble. The camera runs independently of navigation.

1 Start the camera driver

Terminal A: start the driver and keep this terminal running. Do not start a second copy.

```
bash /home/jetson/astra_ws/start_camera.sh
```

2 View the color image in rqt

Terminal B: open another host terminal on the Jetson desktop and run:

```
bash /home/jetson/astra_ws/view_rgb.sh
```

This opens rqt_image_view with the color topic selected. Alternatively, run these two commands in a new host terminal:

```
source /home/jetson/astra_ws/env.sh
ros2 run rqt_image_view rqt_image_view /camera/color/image_raw
```

3 View depth and stop

Keep the driver running and view depth in another host terminal:

```
bash /home/jetson/astra_ws/view_depth.sh
```

Color topic: /camera/color/image_raw. Depth topic: /camera/depth/image_raw. If no image appears, select the topic in the rqt dropdown. Drag the window edge to enlarge a small window.

Press Ctrl+C separately in the driver terminal and each viewer terminal. They are independent processes, so stopping the driver does not automatically close the viewers. No boot autostart is configured.

Source the camera env.sh in every terminal used for manual ros2 commands; it sets ROS_DOMAIN_ID=42. Use separate host terminals for the camera and RTK environments.

UM982 RTK Startup and Data Viewing

Run every command in this section in a Jetson Ubuntu host terminal, outside the navigation container. Workspace: /home/jetson/rtk_ws. Type-C connects to receiver COM3; the stable device name is /dev/rtk_um982, currently at 115200 baud, 8N1.

1 Start the reader

Terminal A: keep the reader running and press Ctrl+C to stop. It has no boot autostart and does not start navigation or configure the receiver mode.

```
bash /home/jetson/rtk_ws/start_rtk.sh
```

2 View status and position

Terminal B: source the RTK environment in every new viewing terminal, then choose a topic:

```
source /home/jetson/rtk_ws/env.sh
```

Combined status, including solution type, satellite count and validity:

```
ros2 topic echo /rtk/status
```

Latitude, longitude and ellipsoid height; stop the previous echo with Ctrl+C first:

```
ros2 topic echo /rtk/fix
```

Other topics: /rtk/heading for dual-antenna heading, /rtk/velocity for ground speed and course, and /rtk/nmea for raw NMEA. View each with ros2 topic echo followed by the topic name.

3 Read the serial port without ROS

Stop the reader in Terminal A to release the port, then use either command below. The first prints parsed data; the second prints raw sentences.

```
bash /home/jetson/rtk_ws/read_rtk.sh
bash /home/jetson/rtk_ws/read_rtk.sh --raw
```

NO_FIX means sentences arrive but there is no position solution, as may occur indoors. NO_DATA means no fresh data; DISCONNECTED means the port is unavailable. quality=4 is RTK fixed and 5 is float. Without a fix, coordinates are NaN and invalid JSON values are null.

env.sh sets ROS_DOMAIN_ID=43. For Permission denied on first use, log out and in or run newgrp dialout first. Outdoor RTK fix, heading and accuracy remain to be tested. The reader is independent of navigation; see the text guide in the workspace for details.